

CSA15 Cloud Computing and Big Data Analytics

CO4 AT2 - Big Data Industry Problem Solving Guided Tool Workbook

Course Code	CSA15	Slot	B
Assessment	CO4 AT2	Bloom Level	BL4 - Analyze
Faculty	Dr C. Rajesh Babu	Employee ID	SSETSCS415
Mode	Individual guided tool-based workbook	Submission	GitHub repository link through Google Classroom
End Evidence	Workbook + screenshots + diagrams + dashboard/mockup	Marks	30

Student Details

Student Name	M.GUNA SHEKAR	Register Number	192472345
Class / Section	C.S.E(AI)	Slot	B
Selected Scenario		Submission Date	24-8-2026
GitHub Link	https://github.com/shekar6193/CSA1512	Google Classroom Status	

Workbook Purpose

What this workbook does

This workbook guides students to use real software tools and official technical references to derive a big-data industry solution. Students should learn from the tool, observe patterns, record evidence and then write their own technical explanation.

- Do not copy full text from websites or AI tools.
- Use official references to justify architecture, processing and security decisions.
- Screenshots are evidence; explanations are the actual assessment answer.

Scenario Bank

Instruction

Faculty may assign one scenario or ask students to complete more than one. If no specific scenario is assigned, select one scenario and complete all tool-guided sections for that scenario.

No.	Scenario	Data Context
1	Healthcare Emergency Analytics	Emergency department data, ambulance updates, bed availability, alerts and treatment delays.
2	E-Commerce Customer Behavior Intelligence	Clickstream events, cart abandonment, payment failure, campaign traffic and recommendation impact.
3	Smart Agriculture Advisory Platform	Weather, soil sensors, crop advisories, pest reports, market prices and farmer app interactions.
4	Cybersecurity Threat Monitoring	Authentication logs, firewall alerts, API access logs, failed login attempts and suspicious network events.
5	Public Transport Demand Analytics	Ticketing data, bus GPS, passenger count, route delay, peak-hour demand and crowding alerts.

Selected Scenario and Reason

Type response / paste screenshot / paste diagram here.

Guided Tool Module 1: Use Cloud Architecture Centers

Purpose: Use official cloud architecture references to identify suitable ingestion, storage, processing, analytics and security components for the selected scenario.

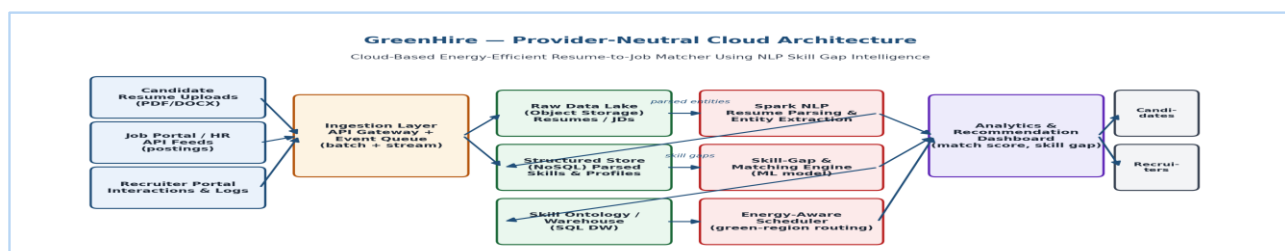
No.	Perform	Observe / Decide	Record in Workbook
1	Open AWS Architecture Center, Azure Architecture Center and Google Cloud Architecture Center in three browser tabs.	You are not copying a full architecture. You are learning common cloud patterns and service categories.	Paste the three page titles or URLs.
2	Search each site using terms from your scenario, such as healthcare analytics, streaming analytics, data lake, retail analytics, IoT analytics or security monitoring.	Notice which patterns repeatedly appear: ingestion, data lake, warehouse, processing, ML/analytics, visualization and security.	Record at least one useful reference from each cloud provider.
3	Identify the services/components mentioned in the reference diagrams.	Separate service category from vendor-specific service name. Example: object storage is the category; Amazon S3/Azure Blob Storage/Cloud Storage are service examples.	Complete the tool-category table.
4	Compare the three references.	Choose the most suitable architecture pattern for your scenario and explain why.	Write a short justification.
5	Draw your own provider-neutral architecture in draw.io.	Your diagram must show data sources -> ingestion -> storage -> processing -> analytics/dashboard -> users, with security controls.	Paste the diagram screenshot/link.
Cloud Provider	Reference Page / URL	Useful Pattern Found	Useful Service Categories
AWS	AWS Architecture Center — Machine Learning & Analytics Lens (aws.amazon.com/architecture/)	Document-ingestion-to-insight pattern: object storage landing zone, event-driven ETL, ML inference, BI dashboard	Object storage, serverless functions, managed NLP/ML, data warehouse, BI/QuickSight-style visualization
Azure	Azure Architecture Center — Intelligent Document Processing pattern (learn.microsoft.com/azure/architecture/)	Form/document intelligence pipeline: ingestion queue → cognitive extraction → knowledge store → Power BI	Blob storage, event/queue ingestion, cognitive/NLP services, data lake, Power BI dashboards
Google Cloud	Google Cloud Architecture Center — Data Analytics / Document AI reference (docs.cloud.google.com/architecture/)	Batch + streaming analytics pattern: Pub/Sub ingestion → Dataflow processing → BigQuery → Looker Studio	Cloud Storage, Pub/Sub, Dataflow (Spark-compatible), BigQuery warehouse, Looker Studio dashboards

Cloud Architecture Justification

Across all three references, the same five-stage pattern recurs regardless of vendor: ingestion → raw storage (data lake) → processing/enrichment → structured storage/warehouse → analytics and visualization, wrapped by a consistent security layer (identity, encryption, monitoring). For GreenHire, the most suitable pattern is the Intelligent Document Processing / Data Analytics pattern common to all three centers, because:

- Resumes and job descriptions are semi-structured documents, matching the document-intelligence pipelines shown by Azure and AWS (cognitive/NLP extraction stage).
- Job-portal feeds and recruiter interactions are continuous events, matching the Pub/Sub → Dataflow streaming pattern shown by Google Cloud.

Architecture Diagram Evidence



Guided Tool Module 2: Use Hadoop and Spark Documentation

Purpose: Decide whether your problem needs batch processing, streaming processing or hybrid processing by reading official Hadoop/Spark documentation.

No.	Perform	Observe / Decide	Record in Workbook
1	Open the Apache Hadoop MapReduce tutorial.	MapReduce is suitable when large stored data can be processed in batches, such as daily logs or historical transactions.	Write one sentence on what mapper and reducer mean.
2	Open the Apache Spark Structured Streaming guide.	Structured Streaming is suitable when new events continuously arrive, such as sensors, clickstream, live alerts or location updates.	Write one sentence on what streaming source, processing and sink mean.
3	Classify your scenario data into batch data and streaming data.	Historical records normally become batch data; live sensor/log/API updates normally become streaming data.	Complete the batch-vs-stream table.
4	Choose one processing approach for your scenario: Hadoop/MapReduce, Spark batch, Spark streaming or hybrid.	The decision must match the data velocity and decision urgency.	Write your processing decision.
5	Prepare a simple pseudo-flow, not full code.	Example: Mapper reads log line -> emits hour, count. Reducer sums counts. Dashboard shows peak hour.	Write pseudo-flow or paste diagram.
Data Item	Batch / Streaming	Why	Suggested Framework
Candidate resume uploads (PDF/DOCX)	Batch	Arrive as discrete documents, not continuous events; parsing is compute-heavy and can tolerate minutes of latency	Spark batch (Spark NLP over Spark SQL/DataFrames)
Historical hiring outcomes (for model training)	Batch	Large, stored, historical dataset used periodically to retrain the matching model	Hadoop MapReduce / Spark batch ETL
Job portal postings feed	Streaming	New postings appear continuously from external HR/job-portal APIs and must be matched promptly	Spark Structured Streaming
Recruiter portal interactions & clicks	Streaming	Live user-interaction events needed for near-real-time dashboard and re-ranking	Spark Structured Streaming
Skill-gap aggregation for dashboard KPIs	Hybrid	Combines nightly batch aggregation with incremental streaming updates as new matches occur	Spark (batch job + streaming job sharing the same Structured APIs)

Processing Framework Decision

GreenHire adopts a hybrid Spark architecture: Spark batch jobs (Spark NLP + Spark SQL) parse resumes and retrain the matching model on a scheduled cycle, while a Spark Structured Streaming job continuously ingests job postings and recruiter events so match scores and skill-gap alerts stay current. Plain Hadoop MapReduce is not used for live matching because of its higher latency, but the same MapReduce mental model (map = extract skill mentions, reduce = aggregate skill demand) is reused conceptually inside the batch Spark jobs. This decision matches the required data velocity: resumes tolerate batch latency, while job-market signals need near-real-time freshness for candidates and recruiters to act on.

MapReduce / Spark Pseudo-Flow

- Mapper reads resume_document → emits (candidate_id, skill_token) for each detected skill entity → Reducer groups by candidate_id → outputs candidate_skill_vector
- readStream(job_postings_topic) → join with candidate_skill_vector (structured stream-static join) → compute cosine similarity match_score → writeStream to match_results_db and dashboard sink
- readStream(match_results) → filter match_score < threshold → compute missing_skills = job.required_skills - candidate.skills → writeStream to alert_feed sink

Guided Tool Module 3: Threat Modeling with OWASP Threat Dragon / Microsoft Tool

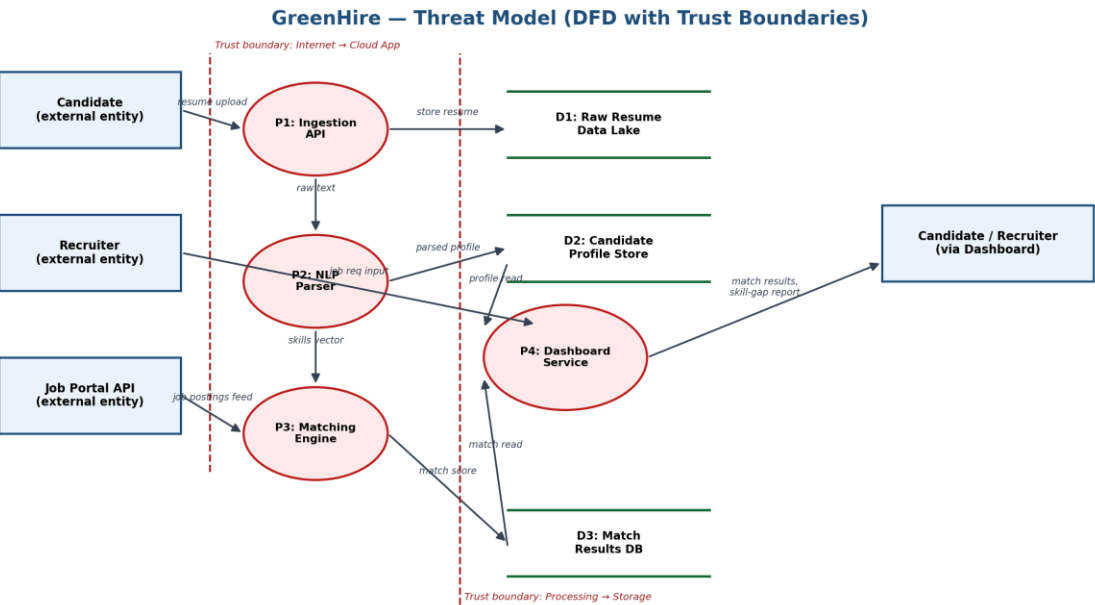
Purpose: Identify privacy and security risks in the big-data solution before proposing controls.

Tool Choice

Use OWASP Threat Dragon if you need a web/desktop cross-platform threat modeling tool. Use Microsoft Threat Modeling Tool when a Windows system is available. The required evidence is the threat model diagram and threat-control table.

No.	Perform	Observe / Decide	Record in Workbook	
1	Open OWASP Threat Dragon or Microsoft Threat Modeling Tool.	A threat model is a security diagram that shows components, data flows, trust boundaries and possible threats.	Write which tool you used.	
2	Create a new threat model named with your scenario.	Use a clear title such as CO4_AT2_Healthcare_Emergency_Analytic s.	Paste model name.	
3	Add external entities.	External entities are users/systems outside your platform: patient, customer, sensor, payment gateway, bus GPS device, public API.	List at least two external entities.	
4	Add processes and data stores.	Processes transform data; data stores hold data. Example: ingestion API, stream processor, data lake, warehouse, dashboard.	List processes and stores.	
5	Draw data flows and trust boundaries.	Trust boundaries mark where security assumptions change: public internet to cloud, app to API, processing layer to storage.	Paste screenshot of DFD.	
6	Identify threats using STRIDE thinking.	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege help structure risks.	Complete the threat table.	
7	Write mitigations.	Controls may include authentication, authorization, encryption, masking, audit logs, rate limits, backup and monitoring.	Map every threat to one control.	
Component / Data Flow		Threat	Impact	Mitigation
Candidate resume upload (Internet → Ingestion API)		Spoofing — attacker submits resumes under a fake identity	Poisoned/duplicate candidate profiles, fraudulent applications	OAuth2/OIDC authentication, email verification, CAPTCHA on upload endpoint
Resume file content		Tampering — malicious payload embedded in PDF/DOCX	Code execution or corrupted parsing pipeline	File-type validation, sandboxed parsing, antivirus/malware scanning before ingestion
Raw Resume Data Lake		Information Disclosure — unauthorized access to PII (name, contact, address)	Privacy breach, regulatory non-compliance	Encryption at rest (AES-256), bucket-level IAM policies, PII tokenization/masking
Ingestion API (public endpoint)		Denial of Service — flooding the upload endpoint	Service downtime, delayed matching for all users	API rate limiting, auto-scaling with quotas, WAF in front of the gateway
Matching Engine → Match Results DB		Tampering — altering match scores to unfairly favor a candidate	Unfair hiring outcomes, loss of trust in the platform	Write-access restricted to service accounts, checksums/audit trail on score updates
Dashboard Service (Recruiter view)		Elevation of Privilege — recruiter viewing another company's candidate data	Data leakage across tenants/companies	Role-based access control, tenant-isolated queries, per-request authorization checks

Threat Model Diagram Evidence



Guided Tool Module 4: Dashboard or Analytics Output Mockup

Purpose: Show how raw big data becomes useful decisions through a dashboard, alert, forecast, recommendation or report.

Beginner-Friendly Choice

Use any one: Power BI, Looker Studio, Tableau Public or Canva. If login/install is difficult, Canva dashboard mockup is acceptable. If data connection is possible, Power BI/Looker Studio/Tableau Public is stronger.

No.	Perform	Observe / Decide	Record in Workbook
1	Prepare a small sample dataset in Excel or Google Sheets with 10-20 rows.	Columns should match your scenario. Example: timestamp, user/location, event type, value, status, risk level.	Paste column names.
2	Choose the dashboard tool.	Power BI/Tableau/Looker are analytics tools; Canva is acceptable for visual mockup when live data connection is blocked.	Write chosen tool and reason.
3	Import or recreate the sample data.	In Power BI/Looker/Tableau, connect data. In Canva, manually create charts from sample values.	Paste screenshot of data/chart setup.
4	Create at least three visual elements.	Use KPI card, bar chart, line chart, table, map, alert count or pie/donut chart as appropriate.	List the visuals.
5	Add decision labels.	A dashboard is useful only if it supports a decision: allocate beds, detect fraud, reroute buses, restock products, warn farmers.	Write decision supported by each visual.
6	Export/share screenshot.	Paste the final dashboard screenshot or public/report link if available.	Paste evidence.

Dataset Column		Meaning	Example Value
candidate_id		Unique identifier for a parsed candidate profile	C-014
job_id		Unique identifier for a job posting	J-231
match_score		Cosine-similarity score between candidate skill vector and job requirements	0.88
missing_skills		Skills required by the job but absent from the candidate profile	Airflow
timestamp		When the match/event was generated	2026-08-20 14:32
decision_flag		Recommended action from the matching engine	Auto-shortlist

Visual Element	Metric Shown	Decision Supported
KPI cards	Avg match score, roles matched, avg skill gaps/candidate, estimated CO2 saved	Quick health check of matching pipeline and green-computing impact
Bar chart	Top skill-gap categories across candidates	Prioritize which upskilling content or training partners to recommend
Line chart	Daily compute energy use: standard vs energy-aware scheduling	Justify keeping/expanding the green-region routing feature
Donut chart	Match score distribution (strong / moderate / weak)	Decide how many candidates need manual recruiter review vs auto-shortlisting
Alert/recommendation table	Per-candidate best-fit role, match %, skill-gap alert, suggested action	Directly drives recruiter shortlisting and candidate upskilling nudges

Dashboard / Analytics Output Evidence

78%

Avg Match Score

14 / 20

Open Roles Matched

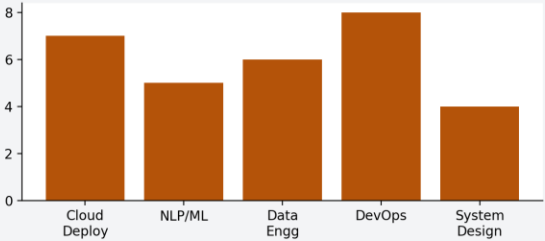
2.3

Avg Skill Gaps / Candidate

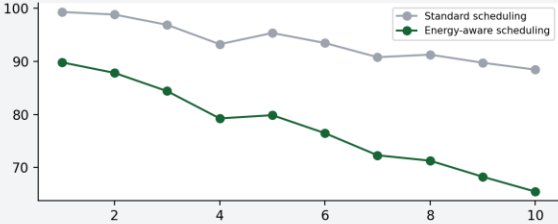
18%

Est. CO2 Saved (green routing)

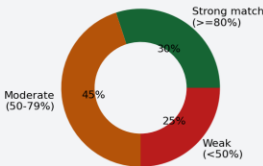
Top Skill Gaps Across Candidates (count)



Daily Compute Energy Use (relative units)



Match Score Distribution



Recommendation / Alert Feed

Candidate	Best-Fit Role	Match	Skill Gap Alert	Decision Supported
C-014	Cloud Data Engineer	88%	Airflow	Auto-shortlist
C-002	ML Ops Engineer	63%	Kubernetes, MLflow	Recommend upskilling
C-019	Backend Developer	42%	System Design, SQL	Suggest alt. role
C-007	NLP Engineer	91%	—	Auto-shortlist

--

Final CO4 AT2 Solution Sections

Final Assembly

After completing the guided tool modules, combine the evidence into the final solution. The final answer must show technical reasoning, not only screenshots.

1. Problem Understanding

What to include: Identify stakeholders, decisions required, data produced, expected analytics output and urgency.

Student Response

- Stakeholders: job-seeking candidates, recruiters/HR teams, the hosting organization's cloud/finance team (energy cost), and platform administrators.
- Decisions required: which candidates to shortlist, which skills to recommend candidates upskill in, and how to route compute jobs to minimize energy/carbon cost.
- Data produced: resume documents, parsed skill vectors, job postings, recruiter click/interaction logs, match scores, and per-job compute-energy telemetry.
- Expected analytics output: ranked candidate-job match scores, skill-gap reports per candidate, and an energy-efficiency dashboard for the platform operator.

2. 5Vs Mapping

What to include: Map Volume, Velocity, Variety, Veracity and Value using scenario evidence.

Student Response

- Thousands of resumes and job postings accumulate continuously across companies; historical hiring records grow the training dataset over time.
- Job postings and recruiter interactions stream in continuously; resumes arrive in batches at variable rates during hiring seasons.
- Unstructured PDF/DOCX resumes, semi-structured job descriptions, structured clickstream logs, and numeric compute-energy telemetry.

3. Big Data Architecture

What to include: Paste final draw.io architecture showing sources, ingestion, storage, processing, analytics/security and users.

Student Response

- the final provider-neutral architecture: data sources (resumes, job feeds, recruiter logs) → ingestion layer (API gateway + event queue) → raw data lake and structured/NoSQL store → Spark NLP processing, matching engine and energy-aware scheduler → analytics/recommendation dashboard → candidates and recruiters, wrapped by a cross-cutting security layer.

4. Data Flow Pipeline

What to include: Explain how raw data moves, gets cleaned, stored, processed and converted into dashboard/alert/decision.

Student Response

- Ingest: candidate uploads a resume or the job-portal API pushes a new posting; the API gateway validates the request and places it on an event queue.
- Land: raw files land untouched in the object-storage data lake for auditability and reprocessing.
- Clean & Parse: Spark NLP extracts entities (skills, experience, education) from resumes and job descriptions, normalizing terms against the skill ontology.

5. Tool and Technology Justification

What to include: Use cloud architecture references and Hadoop/Spark documentation to justify chosen tools.

Student Response

- Object storage (S3/Blob/Cloud Storage) chosen per the AWS/Azure/GCP architecture references as the standard landing zone for unstructured documents — durable, cheap, and decoupled from compute.
- Spark (batch + Structured Streaming) chosen over plain Hadoop MapReduce because GreenHire needs both scheduled batch parsing and continuous streaming matches from a single unified engine, per the official Spark Structured Streaming guide.

6. Security and Privacy Strategy

What to include: Use threat model output to explain threats and controls.

Student Response

Derived from the Module 3 threat model: authentication (OAuth2/OIDC) and file-type validation stop spoofing and tampering at the ingestion boundary; encryption at rest and PII tokenization protect the data lake against information disclosure; API rate limiting and auto-scaling protect availability against denial-of-service; role-based access control and tenant isolation prevent elevation of privilege across the trust boundary between processing and storage; and immutable audit logging of every recruiter and matching-engine action closes the repudiation gap identified in the STRIDE table.

7. Challenges and Solutions

What to include: Write at least four big-data implementation challenges and one practical solution for each.

Student Response

- Use a document-intelligence/NLP service with OCR fallback and a curated skill ontology to normalize synonyms before matching.
- Use Spark Structured Streaming with micro-batch triggers tuned to a few minutes, combined with the energy-aware scheduler to shift non-urgent batch jobs to lower-carbon regions/off-peak windows.

8. Business Outcome

What to include: State measurable outcome: faster decision, reduced fraud, lower waiting time, improved prediction, lower downtime or better user experience.

Student Response

- Faster decisions: recruiters get ranked, explainable shortlists within minutes of a posting going live instead of manually screening resumes.
- Improved candidate experience: candidates receive specific, actionable skill-gap feedback rather than a generic rejection.

Rubric

Criteria	Marks	Evaluation Focus	Evidence
Problem and 5Vs analysis	6	Scenario-specific analysis and correct big-data reasoning	Workbook tables
Architecture and data flow	6	Correct pipeline, cloud components and flow clarity	draw.io diagram
Processing framework decision	5	Batch/streaming/hybrid justification using Hadoop/Spark concepts	Decision table
Security and privacy	5	Threat model, risks and controls	Threat model table/screenshot
Dashboard/output design	4	Useful decision-oriented analytics output	Dashboard/mockup
Clarity and submission quality	4	Organized, original, complete evidence	GitHub/GCR

Submission Checklist

- Student details and selected scenario are filled.
- Cloud architecture references are recorded.
- Hadoop/Spark processing decision is completed.
- Threat model diagram/table is included.
- Dashboard/mockup screenshot or link is included.
- Final solution sections are completed.
- DOCX/PDF and supporting files are uploaded to GitHub.
- GitHub repository link is submitted through Google Classroom.

Official Reference Links

AWS Architecture Center: <https://aws.amazon.com/architecture/>

Azure Architecture Center: <https://learn.microsoft.com/en-us/azure/architecture/>

Google Cloud Architecture Center: <https://docs.cloud.google.com/architecture>

Apache Hadoop MapReduce Tutorial: <https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>

Apache Spark Structured Streaming: <https://spark.apache.org/docs/latest/streaming/index.html>

Microsoft Threat Modeling Tool releases: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-releases>

OWASP Threat Dragon: <https://owasp.org/www-project-threat-dragon/>

Power BI documentation: <https://learn.microsoft.com/en-us/power-bi/>

Looker Studio create report: <https://docs.cloud.google.com/data-studio/create-a-report>

Tableau Public: <https://www.tableau.com/community/public>

Canva Charts: <https://www.canva.com/help/charts/>